

Exercício 1:

A introdução do ponteiro previous permite acesso direto ao penúltimo nó via tail->previous, eliminando a necessidade de percorrer a lista do início. A complexidade passa de $O(n)$ para $O(1)$.

Exercício 2:

O erro é liberar a memória antes de atualizar os ponteiros: resolução:

```
'''C
atual->previous->next = atual->next;
atual->next->previous = atual->previous;
free(atual);
list->size--;
'''
```

Exercício 3:

```
'''C
int buscar_paciente_reverso(DList *list, int id_procurado) {
Node *atual = list->tail;
while (atual != NULL) {
if (atual->id == id_procurado) return 1;
atual = atual->previous;
}
return 0;
}
'''
```

Exercício 4:

```
'''C
novo->next = atual->next;
novo->previous = atual;
atual->next->previous = novo;
atual->next = novo;
'''
```

Exercício 5: A função deve localizar o nó pelo ID, isolá-lo da posição atual ajustando os ponteiros dos vizinhos (sem usar free) e então inseri-lo no Head, atualizando o ponteiro inicial da lista e os elos do novo topo.